

Paradigma de programación estructurada

Es un enfoque de desarrollo que se basa en el uso de estructuras de control bien definidas para mejorar la claridad, calidad y desarrollo del código. En este paradigma, el código se organiza de manera secuencial y estructurada, lo que facilita su comprensión y mantenimiento. **El lenguaje C** es un ejemplo típico de lenguaje que sigue este paradigma.

Principios clave de la programación estructurada en C:

1. Estructuras de control: Se utilizan tres tipos principales de estructuras de control para organizar el flujo de un programa:

- **Secuencia:** Las instrucciones se ejecutan en el orden en que aparecen.
- **Selección:** Se toman decisiones mediante sentencias condicionales como **if, else y switch**.
- **Iteración:** Se repiten bloques de código usando bucles como **for, while y do-while**.

2. Modularidad: El código se organiza en módulos o funciones independientes, lo que facilita la reutilización y el mantenimiento. En C, las funciones permiten dividir el programa en bloques más pequeños y manejables.

3. Evitar el uso del "goto": Aunque C permite el uso de la instrucción **goto**, en la programación estructurada se desaconseja su uso porque puede hacer que el código sea confuso y difícil de seguir.

4. Entrada y salida estructurada: La interacción con el programa sigue flujos controlados, como la lectura de datos con **scanf** o la salida con **printf**, manteniendo la coherencia y simplicidad en el manejo de la entrada y salida de datos.

Ejemplo básico de código estructurado en C:

C

```
#include <stdio.h>

int main() {

    print
    printf("Hola mundo\n");
    return 0;
}
```

Elementos de la sintaxis del programa "Hola mundo":

1. `#include <stdio.h>`:

- Es una **directiva de preprocesador** que le dice al compilador que incluya el contenido del archivo de cabecera `stdio.h` antes de la compilación.
- `stdio.h` es una librería estándar de C que contiene funciones de entrada y salida como `printf`, que se usa para imprimir texto en la consola.

2. `int main()`:

- Es la **función principal** del programa. Todo el programa en C comienza su ejecución desde la función **main**.
- Int** indica que la función **main** devolverá un valor entero al sistema operativo cuando termine.
- Los paréntesis `()` indican que **main** no recibe parámetros (en este caso).

3. `{ ... };`:

- Las llaves `{ }` cerraron el **bloque de código** de la función `main`. Todo lo que esté dentro de estas llaves es el cuerpo de la función, es decir, las instrucciones que el programa ejecutará.

4. `printf("Hola mundo\n");`:

- printf** es una función que pertenece a la librería `stdio.h` y se utiliza para imprimir texto en la consola.
- "Hola mundo\n"** es la cadena de texto que queremos imprimir. Está entre comillas dobles porque es un literal de cadena.
- \n** es un carácter de escape que representa un salto de línea, es decir, cuando el texto se imprime, el cursor de la consola se moverá a la línea siguiente.
- El punto y coma ;** indica el final de la instrucción. Todas las declaraciones en C terminan con un punto y coma.

5. return 0 ;:

- a. La instrucción **return** devuelve un valor al sistema operativo cuando el programa termina.
- b. En este caso, 0 significa que el programa finalizó correctamente. El tipo de retorno de la función **main** es **int**, por lo que debe devolver un número entero.
- c. El punto y coma al final de la instrucción también es necesario.

Resumen de los elementos sintácticos:

- **Directiva de preprocesador (#include)** : Indica qué librerías utilizar.
- **Función main** : Punto de entrada del programa.
- **Bloque de código { }** : Contiene el cuerpo del programa.
- **Instrucción printf** : Imprime una cadena de texto en la consola.
- **Carácter de escape \n** : Representa un salto de línea.
- **return**: Devuelve un valor al finalizar el programa.

***** Programa Ejemplo 1 *****

// #include es una directiva de preprocesador. <stdio.h> es la biblioteca estándar de entrada y salida

// que nos permite usar funciones básicas como printf() para mostrar texto en pantalla.

#include <stdio.h>

// Definición de una función llamada 'imprimirMensaje'.

// 'void' indica que esta función realiza una acción pero no regresa ningún valor de tipo numérico o texto.

void imprimirMensaje() {

 // printf es una función del sistema que imprime en la consola el texto que está entre comillas.

 // El símbolo '\n' genera un salto de línea (un "Enter") en la salida de texto.

printf("Hola, este es un programa estructurado en C.\n");

} // Este cierre de llave indica que aquí termina el bloque de código de la función 'imprimirMensaje'.

// Definición de una función llamada 'sumar'.

// El primer 'int' indica que la función va a devolver un número entero como resultado.

// '(int a, int b)' son los parámetros: la función recibe dos números enteros para trabajar con ellos.

int sumar(int a, int b) {

 // 'return' toma el resultado de la operación (a + b) y lo envía de vuelta a quien invocó la función.

return a + b;

} // Cierre del bloque de la función 'sumar'.

// La función 'main' (principal) es el punto de inicio obligatorio de cualquier programa en C.

// El sistema operativo siempre busca esta función para comenzar a ejecutar las instrucciones.

int main() {

 // Se declaran dos variables de tipo entero ('int') llamadas 'x' e 'y'.

 // Al mismo tiempo se les asignan sus valores iniciales: 5 a la 'x' y 10 a la 'y'.

int x = 5, y = 10;

 // Se declara una variable entera llamada 'resultado'. Esta servirá para guardar el éxito de la suma más adelante.

int resultado;

 // Se invoca (llama) a la función 'imprimirMensaje'. El programa salta a esa función,

 // ejecuta su printf en pantalla y luego regresa aquí para continuar con la siguiente línea.

imprimirMensaje(); // Llamada a función

 // Se llama a la función 'sumar' pasando los valores de 'x' (5) e 'y' (10) como argumentos.

 // El valor devuelto por la función (15) se almacena dentro de la variable 'resultado' mediante el signo '='.

resultado = sumar(x, y); // Llamada a función

 // printf imprime el texto formateado. Cada '%d' es un marcador de posición para números enteros.

 // Los valores de 'x', 'y' y 'resultado' se acomodan en orden en cada uno de los '%d'.

```
printf("La suma de %d y %d es: %d\n", x, y, resultado);

// 'return 0' le avisa al sistema operativo que el programa finalizó correctamente y sin errores.
return 0;

} // Cierre del bloque de la función 'main' y fin del programa.
.....
```

Ventajas del paradigma estructurado en C:

- **Claridad:** El código es más fácil de leer y seguir.
- **Mantenimiento:** La modularidad facilita las actualizaciones y correcciones.
- **Reutilización:** Las funciones permiten reutilizar bloques de código.

En resumen, el desarrollo en el paradigma de programación estructurada en C se centra en un flujo de control claro y el uso de funciones para mejorar la organización y eficiencia del código.